

What happens in a real clinic

CANDIDATE RESPONSE — YAN CHANGHAO — 03 SEP 2026

Every one of these has happened or will and it's not an edge case. Assess these against your own build and overcome them. [We have completed assessing your build for the challenge, so you can modify your repo accordingly.](#)

For each item below, provide the following:

1. **SURVIVES / PARTIAL / DOES NOT** - assess your own build.
2. **Where.** File and line, or the specific absence. "It's handled in the backend" is not an answer.
3. **What breaks first.** If it does not survive, what is the first thing a clinician or patient actually experiences. If it does survive, what could later break it?
4. **Build it better.** Take your existing build or an assigned build and improve it based on the feedback below. See deliverables & timeline below. **Check your email for details.**

We are not looking for a page of green ticks. A candidate who writes *DOES NOT — here is exactly why, and here is what I would change first* is telling us more than one who claims everything holds. Overclaiming is the only answer that counts against you.

1. **The patient has no email.** She exists for the clinic as a phone number in a WhatsApp thread. Can she get into anything you built: intake, portal, summary, instructions? Or does your identity model quietly decide she is not a patient because it relies on email auth alone?

CANDIDATE RESPONSE — DOES NOT

Where: backend/app/schemas.py:1142-1146 and backend/app/api/auth.py:345-387 require an email/password login credential. Patient records may exist without credentials, but there is no phone/WhatsApp authentication or delivery adapter.

What breaks first / next: A phone-only patient cannot register or sign in, so intake, portal summaries and instructions are inaccessible digitally.

Improvement made: Product identity now uses a server session bound to the database User and the patient's own record; the UI does not pretend that WhatsApp access exists.

Remaining limit / first next change: First production change: add verified phone identity/OTP and an approved delivery adapter, plus a clinic-assisted non-digital route. Making email nullable alone would not solve authentication or delivery.

Automated evidence: backend/tests/test_auth_sessions.py; test_auth_invites.py; test_patient_view.py.

2. **One line changes in a route handler.** Name the single place clinic isolation is actually enforced in your build. Now assume that line has a bug. How many patients become visible to the wrong clinic, and what else would have caught it?

CANDIDATE RESPONSE — PARTIAL

Where: The primary enforcement point is backend/app/authz.py:162-181 (authorize_scope -> authorize). Defense in depth is in backend/app/clinic_scope.py and backend/app/db.py:612-1093 (scope indexes and ownership triggers).

What breaks first / next: If an untested read route bypassed both authorize and the scoped loaders, data could leak because SQLite has no PostgreSQL-style row-level security. Tested cross-clinic writes are additionally rejected by ownership triggers.

Improvement made: Added one-query scoped loaders, uniform 404 behavior, active foreign keys, ownership preflight/triggers, scope indexes and an AST bypass gate. A no-op authorize_scope fault is covered.

Remaining limit / first next change: Application isolation is verified for the tested surface, but this is not production multi-tenant certification or database RLS.

Automated evidence: backend/tests/test_fa3_clinic_isolation.py; test_rbac_scope.py; tests/security/test_cross_patient_sentinels.py.

3. Your redaction works. Now read your logs. Grep your application logs, error traces, crash reports and any third-party dashboard for an IC number, a phone number, a patient name. Redaction before the model is the door everyone guards. What is going out the other ones – and for how long is it retained?

CANDIDATE RESPONSE — PARTIAL

Where: backend/app/operational_logging.py:202-251 allowlists, validates and scrubs structured stderr fields. deploy/Caddyfile:4-27 disables access logging and discards edge error logs. Audit rows contain metadata only.

What breaks first / next: Host stderr capture, OS crash dumps, third-party monitoring and Provider-side retention are outside the repository. A deployment could retain data longer than intended even though application logs are sanitized.

Improvement made: Removed raw paths/payloads, added deterministic IC/phone/token scrubbing, suppressed only duplicate handled tracebacks, froze ASR failure codes and added real-process sentinel tests.

Remaining limit / first next change: Retention/purge policy, host log configuration, crash-reporting policy and Provider retention require deployment-owner decisions and external verification.

Automated evidence: backend/tests/security/test_operational_logging.py; test_operational_logging_process.py; test_caddy_access_log_disabled.py; test_caddy_failure_log_sanitization.py; test_voice_storage_and_logs.py.

4. Prove the ordering. Show us the call path where redaction runs *before* the model is invoked. We could verify this in only 12% of submissions.

CANDIDATE RESPONSE — SURVIVES

Where: backend/app/ai_pipeline.py:119-140 loads raw content, calls redact_content at line 129 and only then calls client.summarize at line 140. backend/app/redaction.py:76 performs recursive redaction. backend/app/llm_client.py:57 is the single egress protocol.

What breaks first / next: A future direct SDK/HTTP call outside LLMClient could bypass this ordering; code review and regression tests must keep the single-exit invariant.

Improvement made: Provider APIs accept RedactedContent only; placeholder mapping stays in memory; restoration and exact-span anchoring happen locally and invalid placeholders/quotes fail closed.

Remaining limit / first next change: This proves repository call ordering on synthetic tests, not an external PHI/compliance certification.

Automated evidence: backend/tests/test_redaction.py; test_ai_pipeline_e2e.py; tests/security/test_operational_logging.py.

5. Clinic B onboards next Monday. What breaks? List what you would have to change to serve two clinics, and be specific about what is a config change versus a schema change.

CANDIDATE RESPONSE — PARTIAL

Where: Clinic and settings schema: backend/app/models.py:130-186. Setup flow: backend/app/api/onboarding.py:49-153. Preview/commit import: backend/app/api/patient_imports.py:194-330. Clinic overrides: backend/app/api/clinic_settings.py:78-158.

What breaks first / next: A second clinic can be created without a schema rewrite, but public organisation verification, production provisioning, PostgreSQL RLS, per-clinic Provider billing and support operations are absent.

Improvement made: Added a 24-hour single-use bootstrap, atomic Clinic + first Admin creation, inherited settings, strict UTF-8 CSV preview/commit, stable external IDs, conflict reporting and idempotent repeated/concurrent commit.

Remaining limit / first next change: Config changes: Provider credential, AI/Voice policy and deployment settings. Existing schema supports new Clinic rows. Production would still require a database/platform migration to RLS-backed tenancy and verified organisation administration.

Automated evidence: backend/tests/test_fb5_onboarding.py; test_fb5_clinic_settings.py; test_fb5_patient_import.py; integration/test_fb5_clinic_onboarding_journey.py.

6. The consult is trilingual. Malay, English and Hokkien inside a single sentence, which is the normal case here in SEA, not the edge. What does your transcript produce and what do all the features downstream of that transcript produce?

CANDIDATE RESPONSE — PARTIAL

Where: Manual normalization is in backend/app/transcript_normalizer.py. Doctor review attestation is enforced in backend/app/api/sources.py and the DoctorConsultReviewAttestation schema. Local ASR is in backend/app/voice/asr.py; downstream validation/provenance is in backend/app/ai_pipeline.py.

What breaks first / next: Unicode text is preserved, but local ASR can misrecognize code-switched words and deterministic fallback extraction is mainly English-keyword based. Incorrect transcription propagates unless the clinician corrects it before confirmation.

Improvement made: Added manual speaker-labelled input, continuous indexes, unknown-speaker fail-closed review, three clinician attestations, exact quote anchoring and one synthetic Malay-English-Hokkien Provider evaluation.

Remaining limit / first next change: Current real local-ASR mechanics passed 2/2 but misrecognized two Malay phrases. Live DeepSeek produced 4/4 exact anchors on one synthetic case; neither result is multilingual clinical-validity evidence.

Automated evidence: backend/tests/test_feedback_consult_review.py; test_deepseek_provider_contract.py; test_voice_local_asr_integration.py; test_transcript_normalization.py.

7. A drug allergy is mentioned at minute two of a twenty-minute consult. When does your system know about it? During the consult, or after it ends? Those are two different products with the same feature name.

CANDIDATE RESPONSE — DOES NOT

Where: backend/app/voice and frontend/src/components/VoiceCapture.tsx implement record -> stop -> upload -> transcribe -> human review. There is no streaming-audio endpoint or incremental allergy/risk detector.

What breaks first / next: An allergy mentioned at minute two is not known by Nightingale during the consult. It is available only after recording ends, transcription completes and the transcript is reviewed/confirmed.

Improvement made: The interface labels this as post-consult processing and does not claim real-time alerting.

Remaining limit / first next change: A different product path is required: streaming chunks, incremental transcript state, bounded real-time safety rules, latency metrics and a clinician-facing alert lifecycle.

Automated evidence: backend/tests/test_voice_capture_api_lifecycle.py and test_voice_capabilities.py cover only the declared post-consult boundary.

8. The model hangs. Not an error, a 45-second hang, and the clinician is standing there with the patient. What do they see? 24% of submissions set an explicit timeout.

CANDIDATE RESPONSE — PARTIAL

Where: backend/app/llm_client.py:375-447 applies a 30-second total deadline with asyncio.wait_for, bounded httpx phase timeouts and no retries. backend/app/ai_pipeline.py:140-151 maps ProviderTimeoutError to provider_timeout fallback.

What breaks first / next: After the configured deadline, Consult/Check-in returns a labelled deterministic result instead of waiting 45 seconds. Copilot returns unavailable, and key verification returns 503 without saving the key.

Improvement made: The underlying async request is cancelled; raw source persists before the model call; timeout logs use a fixed metadata-only reason code.

Remaining limit / first next change: Cancellation was verified against a local never-responding HTTP server. A real external DeepSeek timeout was not deliberately induced.

Automated evidence: backend/tests/test_fa5_provider_timeout.py; test_patient_checkin_provider_contract.py.

9. The provider returns 503 for an hour. Does the clinician get something stale and clearly labelled, something rule-derived, or an empty card? 29% of submissions still produce something useful without the model.

CANDIDATE RESPONSE — SURVIVES

Where: backend/app/ai_pipeline.py:140-166 catches unavailable/invalid/timeout results and invokes deterministic_fallback. The raw-first ingestion path is in backend/app/api/sources.py.

What breaks first / next: The clinician receives a clearly labelled deterministic fallback derived from stored source text. The fallback may conservatively produce zero highlights; it never fabricates a source or displays an empty unlabelled AI card.

Improvement made: Provider errors, missing keys, invalid schema and invalid provenance converge on the same server-owned fallback contract while preserving the original input.

Remaining limit / first next change: The fallback is intentionally conservative and English-keyword oriented; it is not equivalent to full Provider understanding.

Automated evidence: backend/tests/test_ai_pipeline_fallback.py; test_source_ingestion_rbac.py; test_doctor_consult_ingestion.py.

10. Two clinicians or a clinician and a staff open the same note at 09:14. Both type. What is in the database at 09:15, and how would either of them know what happened?

CANDIDATE RESPONSE — SURVIVES

Where: backend/app/api/notes.py:194-230 performs atomic compare-and-swap with expected_version and returns 409 for stale writes. Lines 238-341 implement revert and version retrieval. Full snapshots are stored in ArtifactVersion.

What breaks first / next: For the same note, one writer wins and the stale writer receives a visible conflict with the current version. Different role-owned sections are separate Artifacts and do not overwrite each other.

Improvement made: Added full snapshots, diff, revert-as-new-version, metadata-only conflict audit and frontend retry/reload handling.

Remaining limit / first next change: Verified for the current SQLite single-process prototype; no distributed multi-writer certification or real-time cursor presence.

Automated evidence: backend/tests/test_concurrent_edits.py; test_revision_history.py; test_comments.py.

11. Your appointment link generates correctly and the patient never receives it. Why not? Trace the delivery path to the end and name the assumption that fails.

CANDIDATE RESPONSE — PARTIAL

Where: No appointment-link delivery adapter exists. Invite creation produces a one-time link only. backend/app/api/instruction_receipts.py:65-175 implements in-product exact-version Open/Acknowledge receipts for patient instructions.

What breaks first / next: The link never reaches the patient because no email/SMS/WhatsApp sender, delivery receipt, retry, bounce or escalation path exists.

Improvement made: For patient instructions, the UI now says Not viewed / Viewed / Acknowledged and never converts link generation into a false delivery claim.

Remaining limit / first next change: First addition would be an auditable outbound-delivery service with consent, channel preference, idempotency, provider receipt, retry and failure escalation.

Automated evidence: backend/tests/test_b11_instruction_receipts.py; test_auth_invites.py.

12. Your patient-facing summary is wrong. Not hallucinated wildly, but wrong in one dosage. Trace it from generation to the patient's phone. Should there be a human gate, what does it require, and what happens to the copy already sent?

CANDIDATE RESPONSE — PARTIAL

Where: backend/app/api/instruction_publications.py:115-376 implements clinician-only draft/publish/correct/withdraw. backend/app/api/instruction_receipts.py binds patient receipt to artifact_id + version.

What breaks first / next: A generated draft is not patient-visible until a clinician explicitly publishes it. A correction creates a new published revision and preserves the old wording/receipt; a withdrawal hides active portal content without deleting history.

Improvement made: Added a human publication gate, exact-version receipts, superseded lineage, reason-coded withdrawal and audit metadata.

Remaining limit / first next change: Nightingale cannot recall an external screenshot/message because no external delivery channel exists. Medication correctness still depends on clinician review; there is no production medication-reference validator.

Automated evidence: backend/tests/test_b12_instruction_publication.py; test_b12_frontend_contract.py; test_b11_instruction_receipts.py.

13. The nurse recorded a penicillin allergy. The patient told the AI she has no known allergies. Both are in your timeline. What does the clinician see at a glance, and how does the system decide what is seen?

CANDIDATE RESPONSE — SURVIVES

Where: backend/app/conflicts.py:70-227 extracts bounded clinical/allergy assertions. backend/app/ai_pipeline.py:170-210 compares the patient/AI candidate with clinician, staff and confirmed Nurse sources and assigns needs_review.

What breaks first / next: The clinician sees the unresolved allergy contradiction before ordinary Glance suggestions. Both exact sources remain available; the system does not decide which assertion is true.

Improvement made: Expanded authority comparison beyond clinician notes to Staff Notes and confirmed Nurse transcripts; needs_review blocks negative learning and unsafe decay.

Remaining limit / first next change: The assertion parser is bounded English logic and is not a clinically validated allergy NLP system.

Automated evidence: backend/tests/test_ai_conflict_authority.py; test_glance_ordering.py; test_data_decay_policy.py.

14. A risk badge, a confidence label and an importance score are three numbers your interface can render without any of them meaning anything. Pick one you built. What is it, how would we know if it were wrong, and what does the system do when it is?

CANDIDATE RESPONSE — PARTIAL

Where: Importance is a retrieval-order score, not risk probability. backend/app/highlights.py:18-45 defines fixed weights and factor arithmetic; backend/app/glance_projection.py persists role-specific priority_band, final_score and factor_explanation.

What breaks first / next: If the rule ranking is wrong, the clinician can open the exact source, compare surfaced/unsurfaced candidates in Coverage Review and record a quality signal. Formal production order remains deterministic.

Improvement made: Added versioned factors, explicit inclusion/exclusion reasons, stable bands, review labels and hard protection for allergy/conflict/unresolved work.

Remaining limit / first next change: The score is not medically calibrated, and there is no prospective sensitivity/specificity or clinical-outcome study.

Automated evidence: backend/tests/test_fa2_patient_review_workflow.py; test_glance_ordering.py; test_longitudinal_scoring.py; test_sl1_attention_features.py.

15. Your importance ranking learns from clinicians. Only the items you surfaced can be accepted or dismissed, so the loop never sees what it failed to show. It grows more confident about what it already believed. Separately, a care team under a stress load dismisses things they should not. What stops your ranking from learning to bury an allergy because a tired clinician swiped one away on a Tuesday? **7%** of submissions built a learning loop wired to real interaction data. How does this impact trust in the system?

CANDIDATE RESPONSE — PARTIAL

Where: backend/app/importance_learning.py:26-27 caps adjustments and lines 152-205 protect high-risk/review-needed content. backend/app/shadow_learning.py:35 forces base_only serving. backend/app/api/learning.py:152-226 records explicit scoped signals and lines 326-394 provide freeze/rollback.

What breaks first / next: Exposure bias remains because most routine feedback concerns surfaced items, and fatigue/bulk-dismiss behavior is not detected. Therefore a learned model is not allowed to control production Glance.

Improvement made: Recorded content-free decisions for surfaced, unsurfaced and excluded candidates; added Coverage Review, clinic scope, caps, hard protections, synthetic Shadow models, replay, freeze and rollback.

Remaining limit / first next change: No real clinician label volume exists, no fatigue-aware sampling policy is implemented, and no trained model is promoted. Formal serving remains base_only.

Automated evidence: backend/tests/test_sl1_*.py; test_sl2_*.py; test_sl3_*.py; test_self_learning_importance.py.

16. A highlight cites a source that has since been edited. The nurse's note it came from changed yesterday. Does the highlight still resolve, silently point at different text, or vanish? Provenance is either addressable or decorative. Can the system mark dependent output stale and show the original and current versions side by side?

CANDIDATE RESPONSE — SURVIVES

Where: backend/app/provenance_binding.py:38-44 validates source version/hash; lines 66-126 resolve the bound historical snapshot and fail closed on missing snapshot, invalid span or hash mismatch. Highlight fields are persisted by backend/app/ai_pipeline.py:307-322.

What breaks first / next: After the source is edited, the highlight resolves to and verifies the original version and is labelled historical/updated. It never applies the old offset to silently different current text.

Improvement made: Added source_artifact_version, SHA-256 quote binding, idempotent backfill, historical snapshot resolution and current-source mismatch protection for Copilot and learning.

Remaining limit / first next change: The Source reader and version diff expose both states, but there is no dedicated single-screen dependency graph or production archival/disaster-recovery certification.

Automated evidence: backend/tests/test_highlight_provenance.py; test_provenance_resolution.py; test_archive_roundtrip.py; test_phase_e_schema_migration.py.

Overall, the build parts matter. We scored the build to handle the following. Evaluate your repo for the following:

- Streaming real-consult audio and noisy-environment ASR
- Speaker attribution and diarization
- Within-statement code-switching
- Multilingual downstream processing
- Medical terminology and dosage confirmation - should what was captured i.e. medication + dosage be confirmed through medical references and human confirmation?
- Immutable, version-bound provenance
- Fact extraction under negation, correction and conflicting sources

- Real-time collaborative editing without lost updates
- AI regeneration that preserves human-confirmed and completed state
- Explicit handling of contradictory human, patient and AI assertions
- Distinct clinician, staff and patient outputs. Does the system generate according to the audience readability?
- Self-learning that is clinic-scoped, bounded, auditable, resistant to alert fatigue, and evaluated for exposure bias

Capability	Assessment	Current evidence and limit
Streaming real-consult audio and noisy-environment ASR	PARTIAL	Batch local ASR exists; streaming and noisy-clinic validation do not.
Speaker attribution and diarization	DOES NOT	ASR outputs unknown speaker; a human must assign Doctor/Patient. No diarization claim.
Within-statement code-switching	PARTIAL	Unicode transport and review work; one synthetic ASR run misrecognized two Malay phrases.
Multilingual downstream processing	PARTIAL	One live DeepSeek contract/provenance case passed 4/4 anchors; clinical understanding is unvalidated and fallback is mainly English-keyword based.
Medical terminology and dosage confirmation	PARTIAL	Clinician attestation is enforced; no external medication-reference validation exists.
Immutable, version-bound provenance	SURVIVES	Source version + exact span + quote hash resolve the historical snapshot or fail closed.
Negation, correction and conflicting sources	PARTIAL	Bounded tests and correction/conflict workflows exist; this is not validated general clinical NLP.
Real-time collaborative editing	PARTIAL	CAS/409 prevents lost updates and versions/audit explain the outcome; no live co-editing/presence.
AI regeneration preserving confirmed/completed state	DOES NOT	No user-facing AI regeneration workflow is implemented; existing AI paths cannot overwrite human-owned/completed state.
Explicit contradictory assertions	SURVIVES	Allergy and bounded medication/task contradictions preserve both sources and surface needs_review.
Distinct clinician, staff and patient outputs	PARTIAL	Server-enforced projections and authorship are distinct; audience readability has not been clinically evaluated.

Clinic-scoped, bounded, auditable self-learning	PARTIAL	Signals, caps, protection, Shadow replay/freeze/rollback exist; exposure/fatigue evaluation and production serving do not.
---	----------------	--

Deliverables: use form to submit

- 1) Git Repository: working application, automated tests based on scenarios 1-16 above, clear commit history
- 2) README with setup & run instructions (in Git Repo)
- 3) 2-3 Page Technical Brief covering the above 1-17 feedback and how it integrates into your current build. Where you foresee failures. What you tried and got stuck and didn't work. Assumptions you made previously that still stand or don't stand.
- 4) Demo Video: Clearly demonstrate as many 1-16 scenarios as possible.

Git repository: READY LOCALLY — working app, clear commits and scenario-linked automated tests are present. Push the local commits or submit a ZIP before delivery.

README: COMPLETE — setup/run, security/RBAC/redaction, limits and the 1–16 automated-test matrix are included.

Technical Brief: COMPLETE — output/pdf/Nightingale_Technical_Brief.pdf is three pages and records 5 SURVIVES / 9 PARTIAL / 2 DOES NOT.

Demo Video: RECORDED (owner reported) — final full-playback, hash, accessible link and recipient-view check remain candidate-controlled evidence.

Latest verification: 714 backend tests collected: 712 passed and 2 skipped by default; the two explicit local-ASR tests passed when ignored inputs were supplied. Frontend production build passed with 112 modules.

Submission subject: Nightingale 72HR Build — Yan Changhao. Final video URL and repository/ZIP URL must be inserted and access-tested before sending.

Timeline & submission

- **Due:** Thursday, September 3rd 2026, 11:59 PM SGT
- **Submit:** email your **repo link (or zip)**, **brief** and **deliverables** to irakumar@ntngale.com, (cc carrene.teo@ntu.edu.sg)
- Respond to your last **Subject:** Nightingale 72HR Build — <Your Name>